

Counting Problems in Competitive Programming

Math | DP | Combinatorics | Contribution Technique

Rating 800 – 2300 • With Codeforces Practice Problems

Ch	Topic	Rating	Problems included
1	Fundamental Combinatorics	800–1400	Helpful Maths, Fence, Kefa's Park, Flipping Game
2	Inclusion-Exclusion, Stars & Bars	1400–1800	Vasya & Multisets, Derangements, Colorful Graph, Distributing Candies
3	Counting DP — Classic Patterns	1500–2000	Striped, Kboom, Counting Arrays, Beautiful Subsets
4	Contribution Technique	1800–2200	Bro Thinks He's Him, Arpa's Abnormal, Array Beauty, MST Sum
5	Digit DP	1800–2200	Magic Numbers, Salary, Numbers, Round Numbers SPOJ
6	Bitmask DP for Counting	1900–2200	Travelling Salesman, Matching, Coloring, Tourist
7	Burnside's Lemma	2000–2300	Necklace, Color the Fence, Arpa & a Game, Ciel and Gondolas
8	Generating Functions	2100–2300	Partition DP, Ways, Polynomial Counting
9	Master Trigger Table	All	Quick reference — what to reach for

Chapter 1 — Fundamental Counting & Combinatorics

Rating: 800 – 1400

TRIGGER: How many ways to choose / arrange k items from n?

Core Formulas

Formula	Meaning	When to use
$C(n, k) = \frac{n!}{k!(n-k)!}$	Choose k from n, order doesn't matter	Subsets, combinations
$P(n, k) = \frac{n!}{(n-k)!}$	Arrange k from n, order matters	Sequences, permutations
k^n	n independent choices each from k options	Independent decisions
2^n	Number of subsets of n elements	Subset counting
$\frac{n!}{a!b!c!}$	Multinomial: arrange n items with groups	Anagrams, multisets
$C(n+k-1, k-1)$	Stars and Bars: n identical into k bins	Distribution problems

Precompute Factorials (mandatory template)

```
const int MOD = 1e9+7, MAXN = 200005;
ll fact[MAXN], inv_fact[MAXN];

ll power(ll a, ll b, ll mod) {
    ll r=1; a%=mod;
    for(;b>0;b>>=1){if(b&1)r=r*a%mod; a=a*a%mod;}
    return r;
}

void precompute(){
    fact[0]=1;
    for(int i=1;i<MAXN;i++) fact[i]=fact[i-1]*i%MOD;
    inv_fact[MAXN-1]=power(fact[MAXN-1],MOD-2,MOD);
    for(int i=MAXN-2;i>=0;i--) inv_fact[i]=inv_fact[i+1]*(i+1)%MOD;
}

ll C(int n,int k){
    if(k<0||k>n) return 0;
    return fact[n]%MOD*inv_fact[k]%MOD*inv_fact[n-k]%MOD;
}
```

Common Patterns

Pattern	Formula
Grid paths n x m, only right/down	$C(n+m-2, n-1)$
Circular arrangements of n distinct	$(n-1)!$
Pascal identity (key for DP)	$C(n, k) = C(n-1, k-1) + C(n-1, k)$
Vandermonde identity	$C(m+n, r) = \sum C(m, k) * C(n, r-k)$

Practice Problems

Problem	CF / Source	Rating	Key insight
Helpful Maths	CF 339B	800	Count swaps needed — combinatorial observation
Fence	CF 580C	900	Count subsets with given XOR — $2^{(n-1)}$ pattern

Problem	CF / Source	Rating	Key insight
Kefa's Park	CF 580B	900	Tree DFS — count paths, simple multiply
Flipping Game	CF 327B	1000	Fix which 1s to keep, $C(n,k)$ on remaining 0s
Kicker	CF 850B	1100	Count ordered pairs — direct combinatorics
Dreamoon & Sums	CF 476C	1500	Count valid (a,b) pairs — divisibility + $C(n,k)$
Choosing Balls	CF 436E	1700	Count sequences ending in given pair — DP + C

Chapter 2 — Inclusion-Exclusion, Stars & Bars

Rating: 1400 – 1800

TRIGGER: 'At least one', 'none of', 'exactly k satisfy' — conditions over subsets

Inclusion-Exclusion Principle

General form for n conditions:

$$|A_1 \cup A_2 \cup \dots \cup A_n| = \sum \text{over non-empty } S: (-1)^{|S|+1} * |\text{intersection of } S|$$

Problem type	Formula	Rating
Count integers in $[1, n]$ div by at least one of $a_1 \dots a_k$	IE over subsets using LCM for intersections	1400
Derangements — no fixed point	$D(n) = (n-1) * (D(n-1) + D(n-2))$, $D(0)=1$, $D(1)=0$	1500
Surjections: all k bins non-empty	$\sum_{i=0}^k (-1)^i * C(k, i) * (k-i)^n$	1700
Permutations with exactly m fixed points	$C(n, m) * D(n-m)$	1600

Stars and Bars

Condition	Formula
$x_1 + \dots + x_k = n$, $x_i \geq 0$ (identical balls, distinct bins)	$C(n+k-1, k-1)$
$x_1 + \dots + x_k = n$, $x_i \geq 1$ (at least one per bin)	$C(n-1, k-1)$
$x_1 + \dots + x_k \leq n$, $x_i \geq 0$ (at most n total)	$C(n+k, k)$ — add slack variable
$x_i \leq m_i$ (upper bounds)	IE over violated upper bounds

Practice Problems

Problem	CF / Source	Rating	Key insight
Vasya and Multisets	CF 1051E	1700	Count multisets — Stars and Bars with constraints
Distributing Candies	CF 1574D	1600	Count valid distributions — Stars and Bars
Colorful Graph	CF 662C	1700	Count colorings — IE over forbidden pairs
Derangements	CF 501D	1600	Direct derangement formula + DP
Shirts and Bells	CF 894C	1800	Count permutations with forbidden positions — IE
Rabbit Problem	CSES	1600	Stars and Bars with multiple constraints
Count Divisors	CSES	1500	Multiplicative functions + IE on primes

Chapter 3 — Counting DP — Classic Patterns

Rating: 1500 – 2000

TRIGGER: Count the number of ways to build a structure satisfying some property

The Core Idea: $dp[state] = \text{number of ways to reach that state}$

Pattern 1 — Count Subsequences with Property

```
// dp[i] = count of valid subsequences ending at index i
ll dp[MAXN]={0}; ll ans=0;
for(int i=0;i<n;i++){
    dp[i]=1;
    for(int j=0;j<i;j++){
        if(condition(a[j],a[i])) dp[i]=(dp[i]+dp[j])%MOD;
    }
    ans=(ans+dp[i])%MOD;
}
```

Pattern 2 — Count Strings with Adjacent Constraint

```
// dp[i][last] = strings of length i ending in 'last'
// Example: no two consecutive 1s in binary string
ll dp[MAXN][2];
dp[1][0]=dp[1][1]=1;
for(int i=2;i<=n;i++){
    dp[i][0]=(dp[i-1][0]+dp[i-1][1])%MOD;
    dp[i][1]=dp[i-1][0]%MOD; // 1 can only follow 0
}
ll ans=(dp[n][0]+dp[n][1])%MOD; // = Fibonacci F(n+2)
```

Pattern 3 — Two-State DP (Count + Sum together)

When your transition needs both the COUNT of structures and their contribution SUM, track both simultaneously. This is the pattern from 'Bro Thinks He's Him'.

```
// dp_cnt[i] = number of valid subsequences ending at i
// dp_sum[i] = sum of f(b) for all valid subsequences ending at i
ll dp_cnt[MAXN]={0}, dp_sum[MAXN]={0};
for(int i=0;i<n;i++){
    dp_cnt[i]=1; dp_sum[i]=1;
    for(int j=0;j<i;j++){
        if(a[j]==a[i]){
            dp_cnt[i]=(dp_cnt[i]+dp_cnt[j])%MOD;
            dp_sum[i]=(dp_sum[i]+dp_sum[j])%MOD;
        } else {
            dp_cnt[i]=(dp_cnt[i]+dp_cnt[j])%MOD;
            dp_sum[i]=(dp_sum[i]+dp_sum[j]+dp_cnt[j])%MOD;
        }
    }
}
```

Pattern 4 — Knapsack Counting

```
// Count subsets with sum exactly w
ll dp[MAXW+1]={0}; dp[0]=1;
for(int i=0;i<n;i++)
    for(int w=W;w>=a[i];w--) // reverse for 0/1 knapsack
        dp[w]=(dp[w]+dp[w-a[i]])%MOD;
```

Key DP Identities

Structure	Recurrence	Closed form
Binary strings, no two consecutive 1s	$dp[i] = dp[i-1] + dp[i-2]$	$F(n+2)$ Fibonacci
Compositions of n (ordered partitions)	$dp[i] = \sum dp[i-j], j=1..i$	$2^{(n-1)}$
Valid bracket sequences length $2n$	Catalan / ballot DP	$C(2n, n) / (n+1)$
BSTs with n nodes	$dp[n] = \sum dp[i] * dp[n-1-i]$	Catalan: $C(2n, n) / (n+1)$

Practice Problems

Problem	CF / Source	Rating	Key insight
Counting Towers	CSES	1500	Count valid tower constructions — DP with 2 states
Counting Tilings	CSES	1500	Grid tiling — DP on columns (profile DP lite)
Beautiful Subsets	CF 1264D	1600	Count subsets with no two differing by k — DP
Kboom	CF 1197F	2000	Count arrays — DP on sorted + combinatorics
Striped	CF 811E	2000	Count colorings with pattern — DP on segments
Binomial Coefficients	CSES	1400	$C(n, k) \bmod p$ using Lucas / precompute
Count Arrays	CF 461D	1800	Count sequences — DP + $C(n, k)$
LCS of Two Sequences	CSES	1500	Count LCS length AND number of LCS — two-state DP

Chapter 4 — Contribution Technique

Rating: 1800 – 2200

TRIGGER: Sum of $f(\text{subsequence/subarray/subset})$ over ALL such objects — and f decomposes into local terms

The Paradigm Shift

Don't iterate over all objects. Fix a small local element (index, pair, edge) and ask: across how many objects does this element appear? By how much does it contribute each time?

The Adjacent Pairs Decomposition — most important

Whenever f counts blocks / runs / color changes in a sequence:

$$f(b) = 1 + \text{number of positions } k \text{ where } b[k] \neq b[k+1]$$

Summing over all non-empty subsequences:

$$\begin{aligned} \text{Total} &= (2^n - 1) + \text{sum over pairs } (i < j, s[i] \neq s[j]) \text{ of } 2^{(i-1)} * 2^{(n-j)} \\ &\text{Factor as } 2^{(n-1)} * \sum 2^i * 2^{(-j)} \rightarrow \text{segment tree with 5-value node} \end{aligned}$$

Other Common Decompositions

f(object)	Decomposition	Count formula
Sum of all subarray sums	$a[i]$ contributes to subarrays $[l,r]$ containing i	$a[i] * (i+1) * (n-i)$
Sum of subarray maxima	$a[i]$ is max iff no larger element between l and r — monotone stack counts left/right boundaries	$a[i] * L[i] * R[i]$
Number of inversions across all subsequences	Pair (i,j) with $a[i] > a[j]$ is an inversion in $2^{(n-2)}$ subsequences	$\text{inv_count} * 2^{(n-2)}$
Sum of GCDs of all pairs	Use Mobius: $f(d)$ =pairs divisible by d . ans=sum $d * f(d)$ via IE	Mobius inversion
Distinct values in all subarrays	$a[i]$ contributes to $[l,r]$ where $l >$ prev occurrence of $a[i]$	$(i - \text{prev}[i]) * (n - i)$

When Updates Are Added — Wire to Segment Tree / BIT

```
// After deriving contribution formula, handle point updates:
// 1. Before update at k: subtract contribution of k to left pairs
//                               subtract contribution of k to right pairs
// 2. Flip/change s[k]
// 3. Update BIT/segtree at k
// 4. Add new contribution of k to left pairs
// 5. Add new contribution of k to right pairs
// Total: O(log n) per update
```

Practice Problems

Problem	CF / Source	Rating	Key insight
Bro Thinks He's Him	CF (Div2 H)	2200	Blocks in subseqs — adjacent pairs + seg tree (this problem!)
Arpa's Abnormal Array	CF 850C	1900	Sum of lcm of pairs — Mobius + contribution
Array Beauty	CF 1264E	2000	Sum of min diff — contribution + binary search
MST Sum	CF 1245F	2200	Sum of MST edges — contribution via Kruskal + IE
Petya and Array	CF 1187D	1900	Count subarrays with sum in range — contribution
Nastya and Scoreboard	CF 1554F	2200	Sum of subarray contributions — contribution + seg

Problem	CF / Source	Rating	Key insight
Sum of Segments	CF 380C	1600	Sum of all subarray sums — $a[i]*(i+1)*(n-i)$
Sereja and Suffixes	CF 380B	1500	Count contribution of each suffix — direct

Chapter 5 — Digit DP

Rating: 1800 – 2200

TRIGGER: Count numbers in [L, R] satisfying a digit-level property

Answer = $f(R) - f(L-1)$ where $f(X)$ = count of valid numbers in [1, X]

```
// Generic Digit DP – memoised recursion style
string num;
map<tuple<int,int,int,int>, ll> memo;

ll solve(int pos, bool tight, bool leading_zero, int extra) {
    if(pos == (int)num.size()) return !leading_zero && valid_state(extra);
    auto key = make_tuple(pos, tight, leading_zero, extra);
    if(memo.count(key)) return memo[key];

    int lim = tight ? (num[pos]-'0') : 9;
    ll res = 0;
    for(int d = 0; d <= lim; d++){
        bool nt = tight && (d == lim);
        bool nlz = leading_zero && (d == 0);
        int ns = transition(extra, d, nlz);
        res += solve(pos+1, nt, nlz, ns);
    }
    return memo[key] = res;
}

// Call: memo.clear(); f(R)
//      memo.clear(); f(L-1)  <- MUST clear between calls!
// ans = f(R) - f(L-1)
```

Common Extra States

Problem	Extra state	State size
Numbers with digit sum = S	Current sum (0..9*18=162)	$O(18*163*2)$
Numbers divisible by K	Current value mod K	$O(18*K*2)$
No two adjacent equal digits	Last digit placed (0..9)	$O(18*10*2)$
Digit d appears exactly m times	Count of d so far	$O(18*(m+1)*2)$
Monotone non-decreasing digits	Last digit placed (0..9)	$O(18*10*2)$
Count of 1s minus 0s ≥ 0	Difference so far (clipped)	$O(18*40*2)$

Key rule: always clear memoisation between $f(R)$ and $f(L-1)$ calls since the tight flag changes the valid state space.

Practice Problems

Problem	CF / Source	Rating	Key insight
Magic Numbers	CF 628D	1800	Count numbers with alternating even/odd digits
Salary	CF 55D	1700	Count numbers divisible by digit sum — digit DP
Numbers	CF 914C	1700	Count numbers where some digit divides the number

Problem	CF / Source	Rating	Key insight
Round Numbers	SP0J RNDM	1600	Count numbers with #0-bits $\geq$ #1-bits in binary
Digit Count	CF 1202D	1900	Count arrays where digit counts satisfy condition
Fascinating Number	CF 1634E	2100	Count with specific digit occurrences — digit DP
Counting Divisors	CF 1305C	1600	Count numbers in range with specific div count
Counting Numbers	CSES	1500	Count n in $[a,b]$ with no two adjacent equal digits

Chapter 6 — Bitmask DP for Counting

Rating: 1900 – 2200

TRIGGER: Count arrangements of $n \leq 20$ items where state depends on which subset has been used

Core Template

```
// dp[mask] = number of valid arrangements using exactly the elements in mask
// where popcount(mask) = current position being filled
ll dp[1<<MAXN];
dp[0] = 1;
for(int mask=0; mask<(1<<n); mask++){
    if(!dp[mask]) continue;
    int pos = __builtin_popcount(mask);
    for(int i=0; i<n; i++){
        if(mask & (1<<i)) continue;    // element i already used
        if(allowed(mask, i, pos))      // your constraint here
            dp[mask|(1<<i)] += dp[mask];
    }
}
// Answer: dp[(1<<n)-1]
```

Sum over Subsets (SOS) DP

Compute $f[\text{mask}] = \text{sum of } a[\text{sub}]$ for all sub that are subsets of mask. $O(n * 2^n)$ instead of $O(3^n)$.

```
// After initialising f[mask] = a[mask]:
for(int i=0; i<n; i++)
    for(int mask=0; mask<(1<<n); mask++)
        if(mask & (1<<i))
            f[mask] = (f[mask] + f[mask^(1<<i)]) % MOD;
// Now f[mask] = sum of a[sub] for all subsets sub of mask
```

Classic Problems

Problem type	State	Transition
Count Hamiltonian paths	$dp[\text{mask}][v]$ = paths visiting mask, ending at v	$dp[\text{mask} (1 \ll u)][u] += dp[\text{mask}][v]$ if $\text{edge}(v,u)$
Count perfect matchings in bipartite graph	$dp[\text{mask}]$ = ways to match first $\text{popcount}(\text{mask})$ left nodes to subset mask of right nodes	Try each unmatched right node for next left node
Count set partitions into groups of k	$dp[\text{mask}]$ = ways to partition elements in mask	Fix lowest bit, try all k-subsets containing it

Practice Problems

Problem	CF / Source	Rating	Key insight
Counting Hamiltonian Paths	CSES	2000	Classic bitmask DP — $dp[\text{mask}][v]$
Elevator Rides	CSES	1900	Partition people into elevators — bitmask DP
Matching	CSES	2000	Count perfect matchings — bitmask DP
Coloring	CF 553E	2000	Count graph colorings — bitmask + IE

Problem	CF / Source	Rating	Key insight
Tourist	CF 536E	2100	Optimal path through subsets — bitmask DP + merge
Guards in Maze	CF 118D	2000	Guard placement — bitmask + combinatorics
Placing Rooks	CF 1028E	1900	Count rook placements — bitmask DP

Chapter 7 — Burnside's Lemma / Polya Enumeration

Rating: 2000 – 2300

TRIGGER: Count distinct colorings / arrangements under symmetry — rotations, reflections

Burnside's Lemma

Number of distinct objects under symmetry group G :

$$|\text{distinct}| = (1/|G|) * \sum_{g \in G} |\text{Fix}(g)|$$

$\text{Fix}(g)$ = number of colorings unchanged when symmetry g is applied.

Necklace Formula — most common use

Distinct necklaces with n beads, k colors, rotations only:

$$\text{ans} = (1/n) * \sum_{i=0}^{n-1} k^{\gcd(i,n)}$$

Equivalently using Euler ϕ : $\text{ans} = (1/n) * \sum_{d|n} \phi(d) * k^{(n/d)}$

```
ll euler_phi(ll n){
    ll res=n;
    for(ll p=2;p*p<=n;p++){
        if(n%p==0){ while(n%p==0)n/=p; res-=res/p; }
    }
    if(n>1) res-=res/n;
    return res;
}

ll necklaces(ll n, ll k){
    ll ans=0;
    for(ll d=1;d*d<=n;d++){
        if(n%d==0){
            ans=(ans+euler_phi(n/d)*power(k,d,MOD))%MOD;
            if(d!=n/d)
                ans=(ans+euler_phi(d)*power(k,n/d,MOD))%MOD;
        }
    }
    return ans*mod_inv(n,MOD)%MOD;
}
```

Practice Problems

Problem	CF / Source	Rating	Key insight
Necklace	CF 40E	2000	Count distinct necklaces — direct Burnside
Arpa & a Game w Mojtaba	CF 850E	2100	Burnside on permutation game — group theory
Ciel and Gondolas	CF 436E	2200	Burnside + 2D combinatorics
Necklace Counting	CSES	2000	Classic necklace formula with phi
Color the Fence	CF 678F	2300	Burnside on fence colorings with constraints
DNA Evolution	CSES	1800	Count sequences under cyclic symmetry — IE + Burnside

Chapter 8 — Generating Functions & Formal Power Series

Rating: 2100 – 2300

TRIGGER: Multiply independent counting sequences — polynomial convolution to count combined structures

Core Idea

The OGF for sequence $a[0], a[1], \dots$ is $A(x) = \sum a[n] \cdot x^n$. Multiplying two OGFs gives convolution of their sequences — counting ways to combine independent structures.

Key Identities

GF	Encodes	Use
$1/(1-x)$	1,1,1,... (unbounded count)	Unlimited choices per slot
$1/(1-x)^k$	$C(n+k-1, k-1)$ — stars and bars	k independent unlimited choices
$(1+x)^n$	$C(n,0), C(n,1), \dots, C(n,n)$	Subsets of n elements
$(1+x+x^2+\dots+x^m)^k$	Ways to pick sum n using k items each in [0,m]	Bounded choices per slot
$\exp(F(x))$ — EGF	Any number of labeled copies of F structure	Labeled combinatorics

NTT for Fast Polynomial Multiplication

When mod is NTT-friendly ($998244353 = 119 \cdot 2^{23} + 1$), use NTT for $O(n \log n)$ multiply. For other mods use arbitrary-mod NTT (3 NTTs + CRT).

Practice Problems

Problem	CF / Source	Rating	Key insight
Counting Necklaces	CF 898F	2300	EGF for labeled structures — Burnside + GF
Counting Sequences	CF 1065F	2200	Count sequences — OGF multiplication + NTT
Polynomials	CF 958F3	2200	Formal power series operations
Ways to Merge	CF 711E	2300	Counting with GF — partition structures
Integer Partition	CSES	2000	Unordered partition DP — related to GF
Counting Grid	CSES	2200	Grid coloring — GF + IE

Chapter 9 — Master Trigger Table

See the tell. Know the technique. This is the most important page.

When you see...	Reach for...	CF Rating
Choose k from n , arrangements	$C(n,k)$ — precompute fact+inv_fact	800
Grid paths moving only right/down	$C(n+m-2, n-1)$ or simple path DP	900
n identical balls into k distinct bins	Stars and Bars: $C(n+k-1, k-1)$	1200
Count integers in $[1,n]$ coprime to n	Euler's totient $\phi(n)$	1400
Div by at least one of $a_1..a_k$	Inclusion-Exclusion over divisors	1400
Permutations with no fixed point	Derangements $D(n) = (n-1)*(D(n-1)+D(n-2))$	1500
Count strings with local adjacent constraint	DP: $dp[i][last_char]$	1500
Count subsets with sum = X	0/1 Knapsack counting DP	1500
All bins non-empty (surjections)	IE: $\sum (-1)^i * C(k,i) * (k-i)^n$	1700
Count valid bracket / Catalan structures	Catalan number $C(2n,n)/(n+1)$	1700
Count numbers in $[L,R]$ with digit property	Digit DP with tight flag	1800
Sum of f over ALL subarrays, $f = \text{sum/count}$	Contribution: $a[i] * (i+1) * (n-i)$	1700
Sum of subarray maxima/minima	Contribution + monotone stack $L[i]*R[i]$	1900
f involves blocks/runs in subsequence	$f=1+\text{adj_diffs}$, contribution of pair (i,j)	2000
DP works but updates cascade $O(n)$ each	Switch to Contribution + seg tree/BIT	2000
Count paths visiting all $n \leq 20$ nodes	Bitmask DP: $dp[mask][node]$	2000
Subset-sum over all subsets efficiently	SOS DP — $O(n * 2^n)$	2100
Distinct colorings under rotational symmetry	Burnside: $(1/ G) * \sum Fix(g) $	2100
Sum pairs $f(\gcd)$ or $f(\text{lcm})$ over all pairs	Mobius inversion — count by divisor	2100
Combine independent counting groups	Generating Functions — multiply OGFs/EGFs	2200

The most important skill is not memorizing these techniques — it is seeing the TELL that reveals which one applies. Every problem pattern has a tell. When DP cascades under updates — switch to Contribution. When f counts runs in a sequence — decompose into adjacent pairs. When symmetry is mentioned — think Burnside. Train yourself to see the tell by upsolving and asking after every problem: why did the setter choose THIS technique and not another?